Volume xx (20yy), Number z, pp. 1–12

Improving the watertightness of parametric surface/surface intersection

Y. Wang[1,2], X. Jia[1,2], J. Yang[1,2], B. Wang[3], P. Bo[4] and Y. Liu[5]

[1]SKLMS, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, China
[2] University of Chinese Academy of Sciences, China
wangyuqing192@mails.ucas.ac.cn, xhjia@amss.ac.cn, yangjieyin17@mails.ucas.ac.cn
[3] Visual Computing Institute, RWTH Aachen University, Germany
wangbolun@buaa.edu.cn
[4] School of Computer Science and Technology, Harbin Institute of Technology, Weihai, China
pbbo@hit.edu.cn
[5] Microsoft Research Asia, China
yangliu@microsoft.com

Abstract

The parametric surface/surface intersection (SSI) computation serves as a fundamental component in geometric modeling kernels for Computer-Aided Design (CAD) systems. The geometric fidelity of intersection curves — particularly whether the computed intersection loci in the two parametric domains (p-curves) under the two surface maps agree with the true intersection curve in the modeling space — determines the watertightness of the surface trimming. Despite abundant research and industrial developments for SSI algorithms, ensuring the watertightness of the intersection remains challenging, which directly impacts the stability and reliability of the modeling systems.

In this paper, we present a practical algorithm for computing parametric SSI with gap control between the maps in the modeling space of the two p-curves. We first analyze the topology of the two p-curves by solving lower-dimensional systems of equations and build a graph in each domain representing the topological shape. Then we refine the graph through adaptive edge subdivision and construct initial approximate p-curves by interpolation. A constrained optimization framework incorporating distance and tangential information is employed to improve accuracy and minimize gaps. We demonstrate the effectiveness of our algorithm through extensive experiments and comparisons with the intersection package in the open source software OCCT and the commercial engine ACIS.

Keywords: surface intersection, parametric surface, watertightness, topology determination, optimization

CCS Concepts

• **Computing methodologies** → **Parametric curve and surface models;**

1. Introduction

The parametric surface-surface intersection (SSI) is a crucial technique in geometric modeling systems. Given two rational parametric surfaces $\mathbf{F}(u,v)$ and $\mathbf{G}(s,t)$ in 3D space, we take their intersection curve $\mathcal{C}$ as the set of real points

$$\{(x,y,z) \in \mathbb{R}^3 | \exists (u,v;s,t)\ s.t., (x,y,z) = \mathbf{F}(u,v) = \mathbf{G}(s,t)\}.$$

In CAD modeling operations such as editing, analysis, and manufacturing operations, a parametric representation for this intersection curve $\mathcal{C}$ is preferred. Neverthless, an exact parametric form for $\mathcal{C}$ usually does not exist [KS88]. Therefore, several typical algorithms have been developed to give an approximated parametric representation for $\mathcal{C}$, such as the lattice method, the subdivision method, the marching method and hybrid methods. [BFJP87, BK90, LQLX14, JLC22, YJY23]

In CAD systems, 3D models are commonly represented by the Boundary Representation (B-Rep), composed of the geometry information on parametric equations for surfaces and their boundary edges and the topology information on the connectivity among these surfaces. The boundary edges are given by the intersection curves of every two adjacent surfaces in the model.

For the subsequent modeling need, in the B-Rep representation, this intersection curve is stored in three versions: the intersection curve $\mathcal{C}$ in the 3D modeling space, and its two preimage curves $\mathcal{C}_1$ and $\mathcal{C}_2$ in each of the parametric domains of the two surfaces (p-curves). Theoretically, one should have $\mathcal{C} = \mathbf{F}(\mathcal{C}_1) = \mathbf{G}(\mathcal{C}_2)$. How-

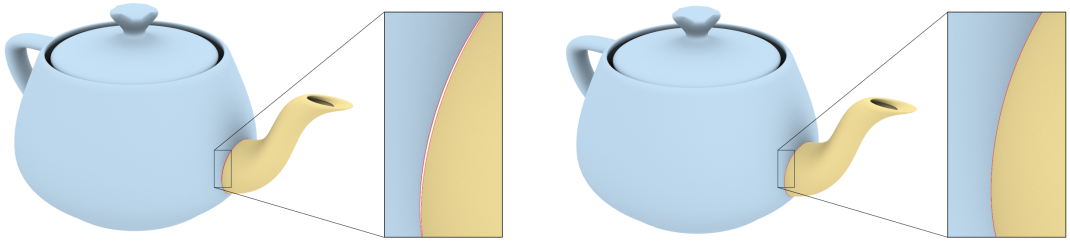


Figure 1: Illustration of a non-watertight model with gaps (left) and a properly sealed watertight model (right).

ever, since usually none of $\mathcal{C}, \mathcal{C}_1, \mathcal{C}_2$ have exact parametric forms, approximate parametric representations are adopted which results in $\mathcal{C} \neq \mathbf{F}(\mathcal{C}_1) \neq \mathbf{G}(\mathcal{C}_2)$, i.e., there exists a gap between the two adjacent surfaces trimmed by the two p-curves. This leads to the so called watertightness problem of modeling, see Fig. 1.

Hence, in B-Rep modeling, a critical target for a practical surface/surface intersection algorithm is to try to minimize the gap between two adjacent trimmed surfaces. One approach employs Hermite interpolation to generate a G^k piecewise parametric approximate of each of the p-curves $\mathcal{C}_1$ and $\mathcal{C}_2$, and reduces the gap by splitting the approximating curves and computing new Hermite interpolation [SN91, HjOwK09a]. Another uses B-spline curves as the approximation, narrowing the gap via control point optimization and knot insertion [KA10, BLZ24].

On the other hand, topological inconsistency of intersection curves can cause improper geometric connections at the junctions of surfaces, such as breaks, crossovers, or non-manifold structures. However, ensuring topological correctness is a challenging task in SSI. The topology of the intersection between two parametric surfaces may exhibit a variety of complicated cases, including tangent points or curves, cusps, self-intersections, and tiny loops [CZXL23]. Conventional methods such as the subdivision method and the tracing method often fail in tackling these singular cases. Therefore, in order to ensure topological correctness, it is preferable to determine the topology of the intersection before approximating its geometric shape. Many works have been conducted on analyzing the topology of parametric surface/surface intersection curves [GK97, HjOwK09b, HFH*07, KA10, CZXL23, YJY23, BLZ24]. However, these methods have certain limitations, including the possible omission of singularities, inefficiency, or reliance on implicitization of curves or surfaces.

In this paper, we present a practical algorithm for computing a B-spline approximation to parametric surface/surface intersection curves in parametric domains. We use topology determination and refinement to generate initial approximate curves and enhance accuracy through constrained optimization. The proposed algorithm demonstrates topological correctness and high accuracy with fewer control points than the commerical geometric modeling kernal ACIS. The main contributions are as follows.

1. The topological shape of p-curves is accurately determined by solving lower dimensional systems of equations, which is extended from algebraic curve topology computation and accelerated by numerical methods.
2. An adaptive refinement technique is designed to control the distance between topological contours and p-curves, facilitating the generation of sufficiently accurate initial curves and thereby accelerating subsequent optimization processes.
3. The optimal approximation to p-curves with relatively few control points is derived from constrained optimization, which directly uses the relationship of the surfaces in intersection, obviating tracing as an intermediate step to generate fitting points.

2. Related work

The parametric surface/surface intersection problem has been studied extensively in recent years. The techniques can be categorized into five groups: the algebraic method, the lattice method, the subdivision method, the marching method, and the hybrid method.

Algebraic methods involve converting one of the parametric surfaces into its implicit form and substituting the parametric representation of the other surface into this implicit equation [Sar83]. This method results in an algebraic curve in the parametric domain, which is the pre-image of the 3D intersection curve. Traditional implicitization methods that use resultants or Gröbner bases suffer from high computational complexity and are impractical in a floating-point environment. Sederberg and Chen [SC95] proposed the technique of moving surface, with Dixon matrices for implicit representation, which has been adopted for SSI problems [KM97, JLC22, YJY23]. Skytt [Sky05] utilized the approximate implicitization method [Dok01] to improve efficiency and applicability. The intersection curves produced by algebraic methods preserve geometric features but often extend beyond the boundaries of parametric surfaces, and their high degree constrains subsequent computational processes.

Lattice methods extract multiple isoparametric curves from one surface and intersect them with the other to approximate the SSI. The intersection curves are obtained by connecting the discrete points [BFJP87]. Tiny loops and isolated points may be neglected under insufficient resolution, and potential errors may arise in the connectivity between points.

Subdivision methods divide the surfaces into smaller patches until every local patch can be approximated by a certain simple surface with sufficient accuracy. Then the intersection is computed between each pair of subpatches. The choice of simple surfaces involves planar patches [HEFS85], bilinear patches [BLZ24], and osculating toroidal patches [PSKE20]. High accuracy intersection computation relies on high level subdivision, which brings rapid memory and computation time growth. Usually hierarchical structures and bounding box collision detection are used to quickly discard the pair of surfaces without intersection to increase efficiency [LQLX14]. Nevertheless, subdivision methods still suffer

from the omission of tiny loops and isolated points, as well as topological errors near singular points.

Marching methods take a set of points on intersection curves as input and generate the complete curves sequentially. In each step, the forward direction and the step length are specified by the local differential geometric information of the surfaces at the given point [BHLH88, BK90]. This method is efficient, but heavily relies on the initial points. Also, determining the forward direction becomes challenging in regions where two surfaces contact or one of the surfaces exhibits self-intersections.

Hybrid methods are developed by integrating two or more approaches, with the objective of leveraging their complementary strengths and mitigating limitations. Dokken et.al [DSY89] introduced a framework of combinging subdivision to detect intersection curves with marching to produce the whole curve. Subsequently, numerous methods have followed this paradigm and dedicated efforts to detecting and handling singularities to enhance robustness [BK90, KPW92, ML95, GK97, ML98, YKK01]. Marching methods are also paired with algebraic methods to ensure topological correctness [KM97, YJY23].

Topological correctness remains the primary focus of most studies that address surface intersection problems. Many of the studies mentioned in the preceding paragraphs serve marching methods, striving to identify all tiny loops and singularities of intersection utilizing normal cones [SM88], gradient fields of oriented distance function between surfaces [ML95, ML98], and Bézier normal vector surface [YKK01]. However, specialized treatments remain essential in the vicinity of the singularities to avoid branch misalignment. Grandine and Klein [GK97] proposed a method (G-K method) that can not only detect a series of characteristic points including singularities but also determine the connectivity between them. With this method, the intersection curve can be generated via solving boundary value problems or Hermite interpolation [HjOwK09a]. Hur et.al [HjOwK09b] supplemented the G-K method by classifying all critical cases that can occur in it and resolving these cases based on the stability of the transversal intersection. The limitations of this type of methods involve the inability to compute the crossing points of intersection curves and the reliance on the sign of floating numbers–derived from geometric information–to determine the topology. Kerber and Sagraloff [KS12] presented another method for topology computation of planar algebraic curves, which has been used in the B-spline surface intersection algorithm [YJY23]. Cheng et.al enumerated all topological cases of parametric surface intersection and ensured topological correctness through surface subdivision and singularity check [CZXL23]. Another work [BLZ24] used the BVH tree combined with lattice methods to compute relatively low-density intersection points and extracted curves via Delaunay triangulation.

The accuracy of the surface intersection, together with the topological correctness, influences the watertightness of the models and the downstream geometric processing and simulation. To align with the formats commonly adopted in the modeling system, intersection curves are expected to be described in B-spline forms. The distance between 3D images of the approximation curves in parametric domains serves as a means to measure accuracy [SN91, KA10, BLZ24]. The first used recursive Hermite interpolation, while the latter two employed curve optimization and knot insertion. Hur [HjOwK09a] developed a novel approach to find the exact maximum discrepancy between the approximation and the actual intersection curves. In addition, several studies focus on localized surface modification in the vicinity of intersection curves, providing an alternative approach to the watertightness problem. Recent advances involve control point perturbation by solving linear equations [SSZ*04], transformation of intersection curves into isoparametric curves [UMC*19], as well as conversion of each trimmed NURBS into an untrimmed T-spline [SFL*08].

3. Method

Our method takes as input two parametric surfaces $\mathbf{F}(u,v)$ and $\mathbf{G}(s,t)$ defined on rectangular domains $\mathcal{P}_1 = [0,1]^2$ and $\mathcal{P}_2 = [0,1]^2$, and a prescribed approximate tolerance ε. The outputs are two sets of planar B-spline curves $\mathcal{B}_i$ contained in $\mathcal{P}_i$ $(i = 1,2)$. Each curve $C_{1r}(\tau)$ in $\mathcal{B}_1$ corresponds to one curve $C_{2r}(\tau)$ in $\mathcal{B}_2$, representing one segment of the intersection and satisfying the precision condition

$$\max_{\tau\in[0,1]} \|\mathbf{F}(C_{1r}(\tau)) - \mathbf{G}(C_{2r}(\tau))\| < \varepsilon. \tag{1}$$

All curves in $\mathcal{B}_i$ collectively form the complete intersection in $\mathcal{P}_i$ $(i = 1,2)$.

Our algorithm consists of four sequential steps, as illustrated in Fig 2. First, a graph is computed in each parametric domain, which is topologically consistent with the intersection and composed of points on the intersection curves and edges between them, as described in Sec. 3.1. Second, the graphs are refined through adaptive edge subdivision guided by local geometric information to control the distance between the topological contours and intersection curves, as described in Sec. 3.2. Third, for each pair of topological contours in two parametric domains, a pair of initial B-spline curves is constructed, with values of control points computed through interpolation and knot vectors assigned proper values to ensure the feasible solution of the interpolation process, as described in Sec. 3.3. Finally, for each pair of curves in two parametric domains, constrained optimization combined with knot insertion is applied to enhance accuracy, as described in Sec. 3.4. The optimization objective function incorporates distance discrepancy of the 3D images of the curves to enforce consistency, and fairness terms to ensure uniform parameterization and avoid excessive distortion. Equality constraints are imposed at the endpoints of the curves to guarantee continuity at the junction of adjacent curves.

3.1. Topology determination

To ensure the topological correctness of the approximate p-curves, it is essential to first analyze and extract the topological shape of the intersection. In this section, we determine the topology of the intersection of $\mathbf{F}(u,v)$ and $\mathbf{G}(s,t)$ in parametric domains $\mathcal{P}_1$ and $\mathcal{P}_2$. The intersection is denoted as

$$\begin{aligned} \mathcal{C}_1 &= \{(u,v) \in \mathcal{P}_1 |\ \exists\ (s,t) \in \mathcal{P}_2, s.t.\ \mathbf{F}(u,v) = \mathbf{G}(s,t)\}, \\ \mathcal{C}_2 &= \{(s,t) \in \mathcal{P}_2 |\ \exists\ (u,v) \in \mathcal{P}_1, s.t.\ \mathbf{F}(u,v) = \mathbf{G}(s,t)\}. \end{aligned} \tag{2}$$

Here we assume that $\mathcal{C}_1$ and $\mathcal{C}_2$ are composed of curve segments and isolated points. Our goal is to compute a graph $\mathcal{T}_i \subset \mathcal{P}_i$, which

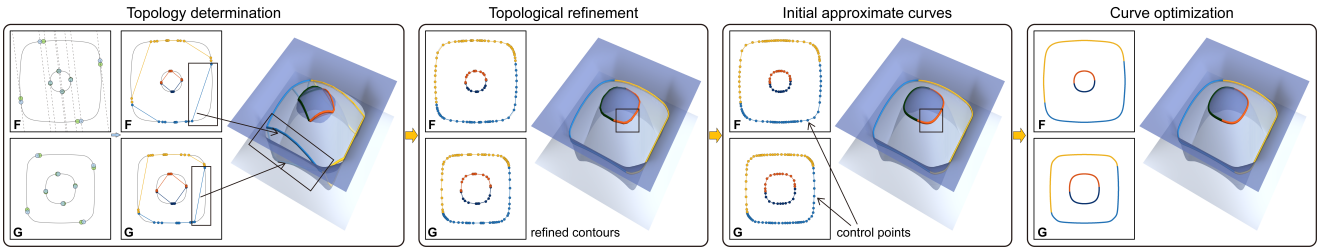


Figure 2: The pipeline of our algorithm. Surfaces **F** and **G** are rendered in dark and light colors, respectively. The 2D plots on the left of each rendering are the parametric domains of **F** and **G**.

represents the topological shape of $\mathcal{C}_i$ $(i = 1,2)$. The vertices of $\mathcal{T}_i$ consist of the undermentioned characteristic points, middle points, and helping points lying on $\mathcal{C}_i$ and the edges depict the connectivity between the vertices. We take $\mathcal{C}_1$ as an example for illustration.

First, we compute three types of characteristic points on $\mathcal{C}_1$ to partition the intersection curves into simple segments. The characteristic points involve boundary points, turning points, and singular points, as shown in Fig 3 and Table 2. A *boundary point* of $\mathcal{C}_i$ is an intersection point of $\mathcal{C}_i$ with the boundary lines of $\mathcal{P}_i$ $(i = 1,2)$, which is computed by solving

$$\begin{aligned} \mathbf{F}(u,v) - \mathbf{G}(s,t) &= 0, \\ u = 0 \text{ or } u = 1 \text{ or } v = 0 \text{ or } v &= 1. \end{aligned} \tag{3}$$

A *turning point* of $\mathcal{C}_1$ with respect to an angle $\theta \in [0,\pi/2)$ is where the tangent direction of $\mathcal{C}_1$ is parallel to $(\sin\theta,\cos\theta)$, which can be obtained by solving the system described in [GK97]

$$\begin{aligned} \mathbf{F}(u,v) - \mathbf{G}(s,t) &= 0, \\ (\mathbf{F}_u \sin\theta - \mathbf{F}_v \cos\theta) \cdot \mathbf{G}_s \times \mathbf{G}_t &= 0. \end{aligned} \tag{4}$$

In practical implementations, we choose a suitable angle θ to avoid failures in topology analysis as described at the end of Sec. 3.1. Meanwhile, we add to $\mathcal{T}_1$ a number of other turning points with respect to $\phi = \pi/2 + \theta$ to split intersection curves into monotonic segments, illustrated in Fig 4. These turning points lie in the middle of certain curve segments of $\mathcal{C}_1$ with characteristic points as endpoints and do not influence the connectivity determination step, which are not regarded as characteristic points and called *middle points* in this paper.

A *singular point* of $\mathcal{C}_1$ is where the tangent direction of $\mathcal{C}_1$ is not uniquely defined [JCY22]. It can be an isolated point, a cusp point, or a self-intersection point. In the first case, the two surfaces touch at the point. It is either a boundary point or an inner tangential point where the normal directions of the two surfaces are parallel, i.e.,

$$\mathbf{F}_u \times \mathbf{F}_v \;/\!/\; \mathbf{G}_s \times \mathbf{G}_t. \tag{5}$$

If the singular point is a cusp point, it can be obtained by solving

$$\mathbf{F}_u \times \mathbf{F}_v = 0 \;\text{ or }\; \mathbf{G}_s \times \mathbf{G}_t = 0. \tag{6}$$

In the last case, if $(u,v) \in \mathcal{C}_1$ is a self-intersection point, there exist $(s_1,t_1),(s_2,t_2) \in \mathcal{C}_2$ and $(s_1,t_1) \neq (s_2,t_2)$ such that

$$\mathbf{F}(u,v) = \mathbf{G}(s_1,t_1) = \mathbf{G}(s_2,t_2). \tag{7}$$

The singular points of the first two cases are included in the solution of Equations 3 and 4, thus the computation of Equations 5 and 6 can be omitted in practice. The computed characteristic points will be used to solve the topology of the intersection curves.

Next, we compute helping points to determine the connectivity of the vertices of $\mathcal{T}_1$. Suppose that the vertices are given by $p_i = (u_i, v_i)$, $i = 0,\ldots,n$. The straight line passing through p_i and parallel to $(\sin\theta,\cos\theta)$ intersects with $\mathcal{C}_1$ at exactly one characteristic point or a middle point p_i. This line can be expressed as

$$l(d_i):\; u\cos\theta + v\sin\theta = d_i, \tag{8}$$

where $d_i = u_i\cos\theta + v_i\sin\theta$ and d_i are distinct for different i. The geometric meaning of d_i is the distance from $l(d_i)$ to the origin. Without loss of generality, assume that p_i are sorted in ascending order of the values of d_i. Thus, $\mathcal{P}_1$ is divided into different panels by the lines $l(d_i)$, and the part of $\mathcal{C}_1$ within the interior of each panel is composed of disjoint curve segments that only meet at certain p_i.

The connectivity between the vertices of $\mathcal{T}_1$ is constructed by sweeping a straight line $u\cos\theta + v\sin\theta = d$ across the parametric domain by increasing d, and detecting the changes in the number of intersection points between $\mathcal{C}_1$ and the line. Practically, let $\mathcal{V}(d_i)$ denote the set of intersection points of $\mathcal{C}_1$ with the straight line $l(d_i)$ and suppose that the points in $\mathcal{V}(d_i)$ are arranged along the positive direction of $(\sin\theta,\cos\theta)$. By definition, $\mathcal{V}(d_i)$ includes p_i. The points in $\mathcal{V}(d_i)$ and $\mathcal{V}(\delta_i)$ except p_i are called *helping points*, where $\delta_i = (d_i + d_{i+1})/2$ $(i = 0,\ldots,n-1)$. With the assistance of the helping points, the graph $\mathcal{T}_1$ is constructed by traversing in the order of increasing i and updating an initially empty 2D list $\mathcal{L}$, with each sublist for one curve segment. For each i, the number of points in $\mathcal{V}(d_i)$ and $\mathcal{V}(\delta_i)$ indicates the emergence of new segments or the termination of existing ones as listed in Table 1, while the index of the characteristic point p_i in $\mathcal{V}(d_i)$ specifies the location of the emergence or termination. Meanwhile, the helping points in $\mathcal{V}(d_i)$ and $\mathcal{V}(\delta_i)$ can be appended to the corresponding segment according to the above information. After adding the points in $\mathcal{V}(d_n)$ and recording all removed segments, we obtain a complete $\mathcal{T}_1$ in the form of a 2D list, with each sublist having characteristic points as its endpoints. The process is illustrated in Fig 3.

Table 1: Distinct branch variations at p_i, with m_i,n_i and N denoting the length of $\mathcal{V}(d_i)$, $\mathcal{V}(\delta_i)$ and $\mathcal{L}$.

$m_i - N$	$n_i - N$	Case	$m_i - N$	$n_i - N$	Case
$+1$	$+1$	start one	$+1$	$+2$	start two
0	-1	end one	-1	-2	end two
-1	0	cross	$+1$	0	isolate
0	0	unchanged	other		error

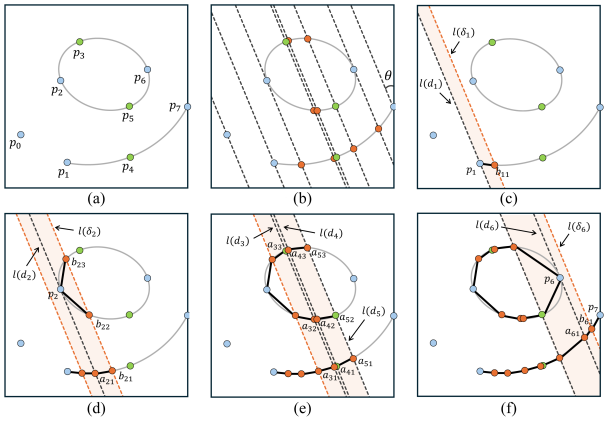


Figure 3: Illustration of the connectivity determination process. (a) shows the characteristic points (blue) and middle points (green). (b) demonstrates how lines $l(d_i)$ partition the domain into distinct panels. (c)–(f) illustrate the sweeping process, where the connectivity is resolved based on the number of helping points (orange).

Table 2: Classification and functions of points used in topology determination, with notation corresponding to the symbols in Fig 3.

Notation	Category		Function
p_0	singular	charac-teristic	split curves into simple segments
p_1, p_7	boundary		
p_2, p_6	turning		
p_3, p_4, p_5	middle		split segments into monotonic parts
a_{ir}, b_{ir}	helping		determine connectivity

This method for determining the connectivity of the points in each panel is similar to [KS12], while we use general slant lines rather than vertical lines to divide the domain, ensuring that there is exactly one characteristic point p_i on each line l_i: If there are two characteristic points p_i and p_j such that $d_i = d_j$, the width of the panel bounded by the two lines is 0, which makes the connectivity analysis fail. In this case, we randomly select another θ to avoid failures. Remark that due to the correspondence of $\mathcal{C}_1$ and $\mathcal{C}_2$, once we computed $\mathcal{T}_1$, $\mathcal{T}_2$ can be readily obtained. When computing the vertices of $\mathcal{T}_1$, we also compute the singular points and turning points of $\mathcal{T}_2$ with respect to $\bar{\theta}$ and $\bar{\phi}$ and regard them as middle points. The purpose is to partition $\mathcal{T}_2$ into monotonic segments and make it satisfy the preconditions for the subsequent topological refinement phase. The $\bar{\theta}$ and $\bar{\phi}$ here do not influence the topology computation process, so they can be chosen arbitrarily. In practical implementation, we set $\bar{\theta} = \theta$ and $\bar{\phi} = \phi$, the same as $\mathcal{T}_1$.

The topology determination step involves solving a series of nonlinear systems of equations. To balance efficiency and robustness, we employ the numerical method. Concretely, for initialization, we first discretize the surfaces into triangular meshes and construct kdtrees to enable efficient spatial querying. Then we sample the parametric domain boundaries and the straight lines $l(d_i)$ for the optimization of boundary points and helping points, and sample within the interior of the domain for turning points and singular points. The closest point on the other surface to a given sample point can be efficiently retrieved by using the kdtree, and this pair of points is used as initial values for the optimization. We then use Newton iteration to obtain the accurate solutions. We select the default sampling density as 15×15. If unexpected cases occur, we increase the density to avoid failures caused by insufficient sampling points. The algorithm is outlined in Algorithm 1.

Algorithm 1 Topology determination

Input: parametric surfaces $\mathbf{F}(u,v)$ and $\mathbf{G}(s,t)$;
Output: topological shape of the intersection;

choose a random θ;
compute characteristic points and middle points $\{p_i\}_{i=0}^{n}$;
sort $\{p_i\}_{i=0}^{n}$ in ascending order of d_i;
initial an empty 2D list $\mathcal{L}$, with N denoting its current length;
for $i = 0, 1, \cdots, n-1$ **do**
    compute $\mathcal{V}(d_i) = \{a_{ir}\}_{r=1}^{m_i}$;
    if p_i is a middle point **then**
        if $m_i \neq N$ **then**
            increase sampling density and go back to 2;
        end if
        append a_{ir} to the corresponding sublist l_i of $\mathcal{L}$;
    else
        compute $\mathcal{V}(\delta_i) = \{b_{ir}\}_{r=1}^{n_i}$;
        determine topo cases based on m_i, n_i and N;
        if unexpected cases occur **then**
            increase sampling density and go back to 2;
        end if
        insert sublists or remove and record sublists;
        append a_{ir} and b_{ir} to the corresponding sublist;
    end if
end for
append p_n to each sublist of $\mathcal{L}$;
return the recorded sublists;

3.2. Refinement of the topological contours

After topology determination, we get $\mathcal{T}_1$ and $\mathcal{T}_2$, which are made up of topological contours and depict the topological shape of $\mathcal{C}_1$ and $\mathcal{C}_2$. However, the distance between the intersection curves and their topological contours can be substantial in the parametric domain, as shown in Fig 4. Since the initial curves used for subsequent optimization are constructed from topological contours, a large distance can make the initial curves far from the optimal solution, thereby limiting the effectiveness and efficiency of optimization. Therefore, in order to facilitate the optimization process, in this section, we aim to adaptively refine the topological contours of $\mathcal{T}_1$ and $\mathcal{T}_2$ based on a prescribed threshold δ. Here we take $\mathcal{T}_1$ as an example for illustration and the same operation can be done on $\mathcal{T}_2$.

For any two adjacent vertices p_i and p_{i+1} in $\mathcal{T}_1$, let c_i be the intersection curve segment with endpoints p_i and p_{i+1}. For any two points $p_{i,r-1}$ and $p_{i,r}$ on c_i, let $c_{i,r}$ be the intersection curve between the two points. The topology determination process ensures that $c_{i,r}$ is monotonic, thereby bounded by the rectangle with one pair of opposite sides parallel to $(\sin\theta, \cos\theta)$ and $p_{i,r-1}p_{i,r}$ as its diagonal.

Therefore, the distance between $c_{i,r}$ and the edge $p_{i,r-1}p_{i,r}$ is less than half the edge length.

Based on this fact, the topological contours is refined recursively until the length of any edge is less than the threshold δ. The geometric accuracy of the topological contours increases as δ decreases, thereby providing better initial approximate curves for the subsequent optimization process in Sec. 3.4. This topological contour refinement process can be incorporated into the topology determination step in Sec. 3.1. When traversing the panels, we compute the distance between each to-be-added point and the last-added point on the same curve. If the distance exceeds δ, we insert a new line in this panel and add the intersection points between $\mathcal{C}_1$ and the line to each segment. The benefit of simultaneous refinement during topology calculation is that it is easier to determine the segment in which the refining point should reside.

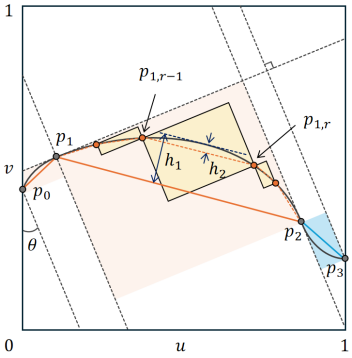


Figure 4: An example where the maximum distance from the p-curve (black) to its topological contour (orange and blue for different segments partitioned by characteristic points) is h_1. After inserting the orange points to refine the edge p_1p_2, the maximum distance decreases to h_2.

3.3. Construction of initial approximate curves

Now we have two sets of refined topological contours in two parametric domains $\mathcal{P}_1$ and $\mathcal{P}_2$. The topological contours in the two sets are in one-to-one correspondence, and their endpoints have the same images in 3D space. Each pair of topological contours corresponds to one pair of segments of p-curves with the same endpoints in $\mathcal{P}_1$ and $\mathcal{P}_2$ separately. In this section, based on the refined topological contours, we construct B-spline curves as an initial approximation to the p-curves for subsequent curve optimization, using the traditional interpolation technique [PT96, Boo01] .

Suppose that a pair of topological contours of $\mathcal{T}_1$ and $\mathcal{T}_2$ is composed of sequential vertices p_i and q_i, $i = 0,\ldots,m$. Cubic B-spline curves $C_1(\tau)$ and $C_2(\tau)$ with $m+1$ control points will be determined to act as initial curves. We first assign parameters $\bar{\tau}_i$ to p_i and q_i using the chord length of 3D points $\{\mathbf{F}(p_0),\ldots,\mathbf{F}(p_m)\}$ or $\{\mathbf{G}(q_0),\ldots,\mathbf{G}(q_m)\}$. Then the knot vector is set to $\{0,0,0,0,\tau_4,\ldots,\tau_m,1,1,1,1\}$, where $\tau_{i+3} = \frac{1}{3}(\bar{\tau}_i+\bar{\tau}_{i+1}+\bar{\tau}_{i+2})$. The values of control points are obtained by solving the following linear systems

$$C_1(\bar{\tau}_i) = p_i, \qquad i = 0,\ldots,m, \tag{9}$$
$$C_2(\bar{\tau}_i) = q_i, \qquad i = 0,\ldots,m. \tag{10}$$

The existence and uniqueness of the solution are guaranteed by the Schoenberg-Whitney condition [Boo01]. Note that the initial curves we generate have the same endpoints as the corresponding topological contours and p-curve segments. Moreover, the two curves have identical parameters at their respective interpolation points, which supports the isoparametric representation of the curves and enhances their geometric consistency.

3.4. Optimization of approximate curves

For each pair of p-curve segments, we have obtained a pair of initial approximate B-spline curves

$$\begin{aligned} C_1(\tau) &= (u(\tau),v(\tau)) = \sum_{i=0}^{m} P_i B_{i,3}(\tau), \\ C_2(\tau) &= (s(\tau),t(\tau)) = \sum_{i=0}^{m} Q_i B_{i,3}(\tau), \end{aligned} \tag{11}$$

that are defined on the knot vector $\{r_0,\ldots,r_{m+4}\}$, where $P_i \in \mathcal{P}_1$ and $Q_i \in \mathcal{P}_2$ are the variables to be optimized. To achieve consistency of approximate curves, in this section, we minimize the distance between spatial curves $\mathbf{F}(C_1(\tau))$ and $\mathbf{G}(C_2(\tau))$ and use knot insertion to increase the degree of freedom for optimization. In practical implementation, we alternately optimize P_i and Q_i. Suppose that we first fix Q_i and optimize P_i.

To evaluate the distance between $\mathbf{F}(C_1(\tau))$ and $\mathbf{G}(C_2(\tau))$, we uniformly sample M values in each knot interval $[r_k, r_{k+1}]$, $k = 3,\ldots,m$ and obtain $N = M(m-2)$ pairs of points $C_1(\tau_i)$ and $C_2(\tau_i)$, $i = 1,\ldots,N$. Then the distance function is represented as

$$E_{dis} = \sum_{i=1}^{N} \|\mathbf{F}(C_1(\tau_i)) - \mathbf{G}(C_2(\tau_i))\|^2. \tag{12}$$

The topological analysis procedure ensures that each segment of p-curves we aim to approximate here is free of self-intersections and entanglements, which means the geometric shapes of C_1 and C_2 should be relatively simple. Therefore, we add the fairness term to avoid topological changes in the optimization process. The fairness term with respect to C_1 is

$$E_{fair} = w_1 \int_0^1 \|\dot{C}_1(\tau)\|^2 d\tau + w_2 \int_0^1 \|\ddot{C}_1(\tau)\|^2 d\tau, \tag{13}$$

where w_1 and w_2 are weights, initially set to $w_1 = w_2 = E_{dis}/E_{fair}$ and halved in each iteration. When w_1 and w_2 are less than 10^{-5}, we discard the fairness term so that the approximate curves can better capture the subtle features of the exact p-curves.

Combining the distance function and the fairness term, we obtain the complete objective function as

$$E = E_{dis} + E_{fair}, \tag{14}$$

with the bounds of variables $P_i \in \mathcal{P}_1$. And to ensure the continuity between different approximate curve segments, we add equality constrains at endpoints. Suppose that the p-curve segment to be approximated has end points α_0 and α_1, where the unit tangent vectors are β_0 and β_1. The G^1 continuity condition is expressed as

$$\begin{aligned} C_1(0) &= \alpha_0, & C_1(1) &= \alpha_1, \\ \dot{C}_1(0) &= \lambda_0\beta_0, & \dot{C}_1(1) &= \lambda_1\beta_1. \end{aligned} \tag{15}$$

The first two equations require $P_0 = \alpha_0$ and $P_m = \alpha_1$, and the latter two equations require $P_1 - P_0 = \lambda_0\beta_0$ and $P_m - P_{m-1} = \lambda_1\beta_1$, where λ_i are numerical values to be optimized and β_i are

pre-computed using surface geometric information. These equality constraints can be substituted directly into the objective function.

Although E_{fair} is quadratic in P_i, the degree of E_{dis} with respect to P_i equals the sum of degrees in the u and v directions of $\mathbf{F}(u,v)$, which means that the objective function might be highly non-linear in P_i. Benefit from the topological contour analysis and refinement, the approximate curves are close to the actual p-curves from the initial iteration of optimization. Therefore, we perform the Taylor expansion of E_{dis} and obtain

$$\tilde{E}_{dis} = \sum_{i=1}^{N} \|\mathbf{F}(C_1(\tau_i)) + \nabla\mathbf{F}(C_1(\tau_i)) \cdot \Delta C_1(\tau_i) - \mathbf{G}(C_2(\tau_i))\|^2, \tag{16}$$

where $\Delta C_1(\tau) = \sum_{i=0}^{m} \Delta P_i B_{i,3}(\tau)$ and ΔP_i are the incremental values of P_i $(i = 0,\ldots,m)$. Then the objective function can be modeled as a quadratic function of P_i

$$\tilde{E} = \tilde{E}_{dis} + E_{fair}, \tag{17}$$

and the optimization problem can be solved as linear least-squares.

The approximation error of C_1 and C_2 is measured by

$$e_{\max} = \max_{k,i} \|\mathbf{F}(C_1(\tau_{k_i})) - \mathbf{G}(C_2(\tau_{k_i}))\|^2, \tag{18}$$

where $\{\tau_{k_i}\}$ are $\tilde{M}$ uniformly sampling values in each knot interval $[r_k, r_{k+1}]$, $k = 3,\ldots,m$. We set the value of $\tilde{M}$ larger than M, to obtain a more comprehensive assessment of the optimization result.

In each iteration, we compute the approximation error $e_{\max}$ with the corresponding parameter $\tau_{\max}$ and the maximum incremental value of P_i and Q_i $(i = 0,\ldots,m)$, and terminate the optimization loop if $e_{\max}$ is less than the approximation tolerance ε or the incremental value is less than a predefined threshold $\tilde{\delta}$. If optimization terminates but $e_{\max} > \varepsilon$, it indicates an insufficient number of control points to approximate the exact p-curves. Then, we find the knot interval involving $\tau_{\max}$ and insert a new knot in the middle of this knot interval. The optimization-insertion process will continue until $e_{\max} < \varepsilon$ or the maximum iteration limit is reached.

4. Implementation

To ensure reproducibility, we provide implementation details of our algorithm, including parameter configurations, external library dependencies, and optimization strategies. The algorithm is implemented in C++, with step-by-step details as follows.

The topology determination step follows the numerical implementation detailed in Sec.3.1, with specific implementation details: θ is randomly chosen within $[0, \pi/2]$; kdtree construction and spatial querying utilize the CGAL library [The25]; initial sampling density is set to 15×15 with adaptive increases triggered by error detection; and Newton iteration is implemented in-house.

The refinement step is incorporated into the topology determination step as described in Sec. 3.2. The points inserted are computed using the same method as the helping points detailed in Sec. 3.1, with the refinement threshold δ set to 0.05.

The initial curve construction step follows the approach outlined in Sec. 3.3, with control points solved using the colPivHouseholderQr solver from the Eigen library.

The approximate curve optimization step follows the optimization-insertion process detailed in Sec. 3.4. The objective function $\tilde{E}$ is a quadratic function of variables; the optimal solution is determined by taking the derivative of the objective function and solving the resulting system of linear equations using Eigen's colPivHouseholderQr solver. The parameter settings are as follows: sampling density $M = 20$; fairness weights w_1 and w_2 derived from the initial curves and halved in each iteration; error evaluation sampling $\tilde{M} = 50$; and incremental threshold $\tilde{\delta} = 10^{-5}$.

5. Results and comparison

We implemented our algorithm and performed comparisons with the topological structure extraction method based on the BVH tree and Delaunay triangulation (BVH-D) [BLZ24], the intersection algorithms in the representative open-source software OCCT [SAS25], as well as the commercial 3D Modeler ACIS [Tea25]. All experiments are executed on a PC equipped with an Intel Core i7-14700 2.10 GHz CPU and 16 GB RAM. The results collectively demonstrate our advantages in enhancing topological correctness and achieving higher precision with fewer data points.

5.1. Topological correctness

We first concentrate on the topology correctness of the parametric surface/surface intersection. A series of surface intersection cases with diverse topologies is constructed for experiments, including self-intersections, tangent points and curves, cusps, tiny loops, and hybrid cases. Nine typical examples and the intersection results in two parametric domains computed by four algorithms are presented in Fig 5, in which our method always gives the correct topology, while the other three algorithms give incorrect or incomplete results in certain cases. The empty figure represents that there are no intersection results in the corresponding parametric domain and "N/A" means the result exceeds the domain.

Our algorithm succeeds in all these topologically singular cases because our topology determination step establishes correct correspondences between the intersection curves in the two parametric domains. Although we solved the characteristic points through sampling and numerical optimization methods, the error-detection operation combined with recalculating with a higher sampling density enhanced the reliability of our method. BVH-D produces incomplete results in examples (3)(4)(9) and gives intersection results with incorrect connectivity in examples (1)(2)(5)(6)(8), showing spurious adjacencies in examples (1)(2)(6) and missing connections in examples (5)(8). Analysis of the intermediate results of BVH-D reveals that the uneven distribution of computed points might be the primary cause of segment omissions and spurious connections. This imbalance creates ambiguities in defining topological connectivity and requires the algorithm to establish consistent adjacency rules across varying point densities. As a result, selecting appropriate topological parameters becomes highly challenging, often requiring case-specific tuning that harms generalizability. OCCT yields partial intersection results in examples (3)(4)(7) and no intersection in example (6). The loss of tiny loops, isolated points, and tangential curves in OCCT might be attributed to its use of discrete methods for intersection detection. ACIS successfully

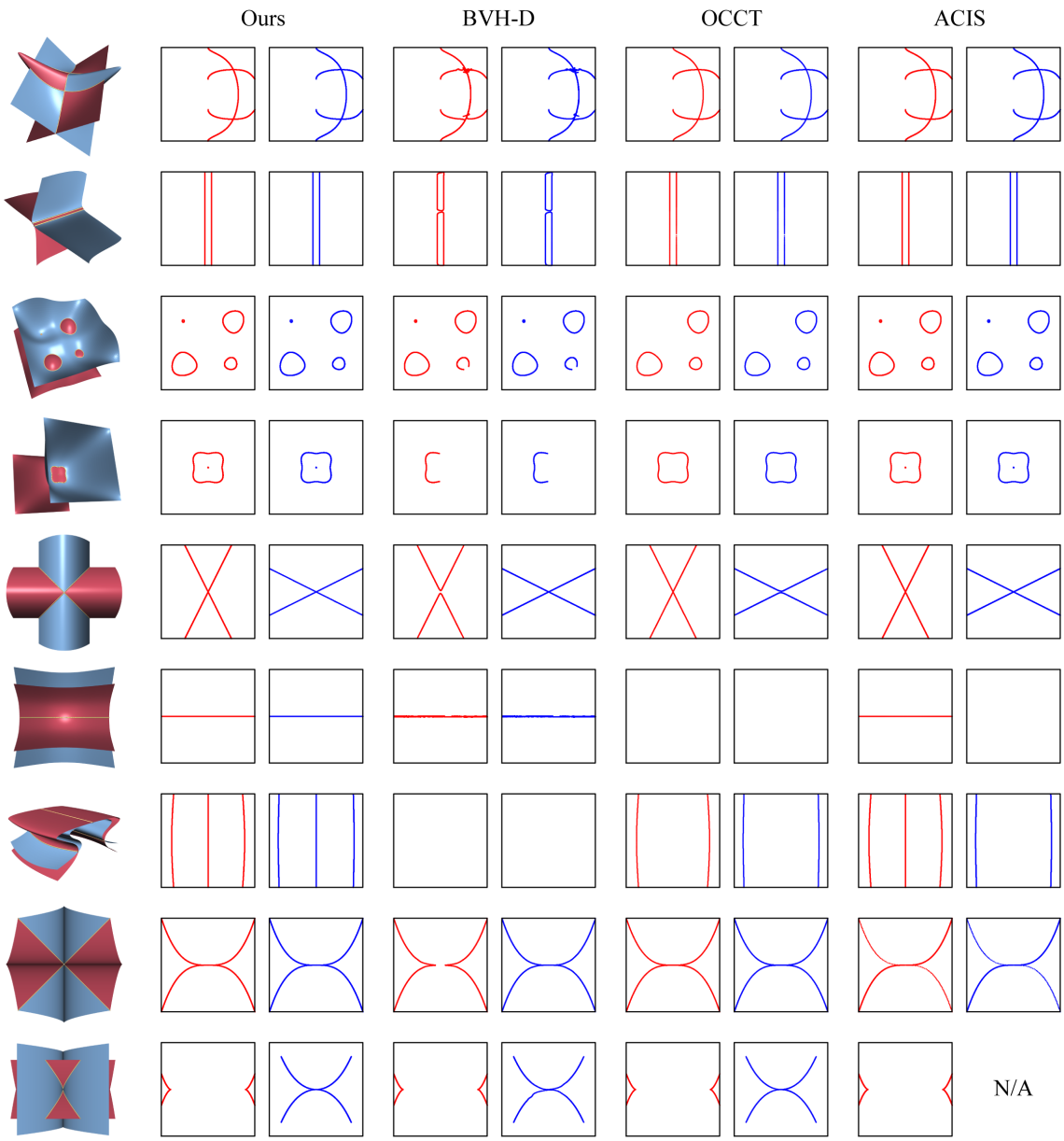


Figure 5: Typical examples with intersection curves of special topology in parametric domains. The first column shows the surfaces and the intersection curves. For each of the 4 comparison methods, the p-curves generated are plotted in red and blue in the 2 parametric domains. N/A means that the results are generated but out of the parametric domain, indicating solving failures.

computes accurate 3D intersection geometries for all test cases but gives undesired outputs in the second parametric domain in examples (6)(7)(9). Tracing and interpolation techniques are employed to derive B-spline approximate intersection curves, and subdivision is adopted to enhance accuracy. Since the 3D intersections are topologically and geometrically correct, the observed failures are likely attributed to subsequent steps in parametric curve generation, potentially influenced by tangential points or curves.

5.2. Accuracy and efficiency

We tested on various parametric surfaces with different types of intersection, as shown in Fig 6 and Table 3. As the approximate tolerance decreased progressively from 10^{-4} to 10^{-8}, our algorithm consistently delivered results satisfying the tolerance criteria within a computationally practical time span. The intersection function *d3_sf_sf_int* of ACIS generates a list of objects *curve* that allows multiple operations, such as evaluating the 3D positions and tangent vectors for each input curve parameter, and a list of B-spline curves *intcurve* that interpolates the points sampled from

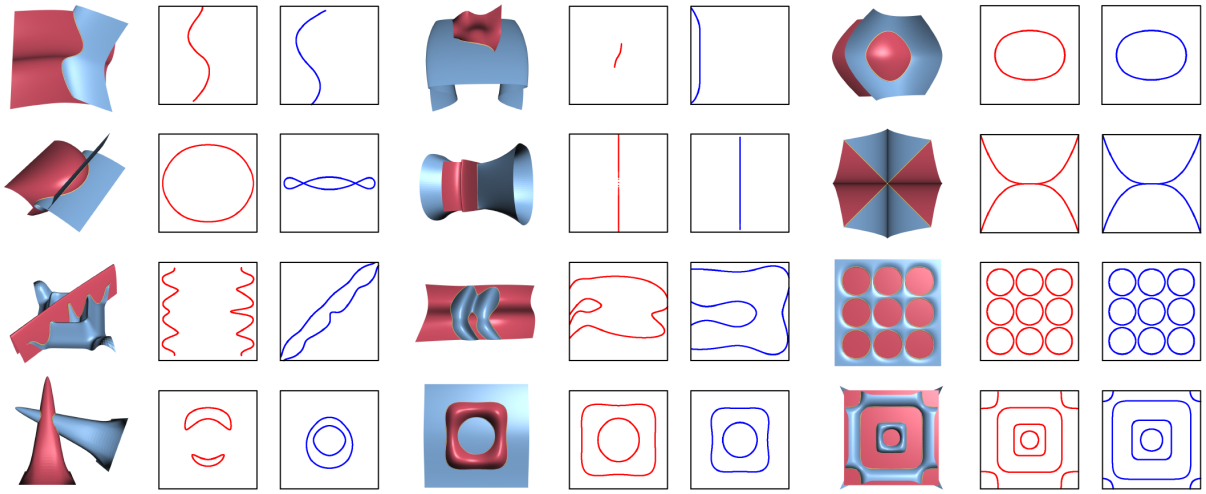


Figure 6: Parametric surface/surface intersection examples employed for comparing the accuracy and runtime of our method, ACIS, and OCCT, with the statistical results listed in Table 3.

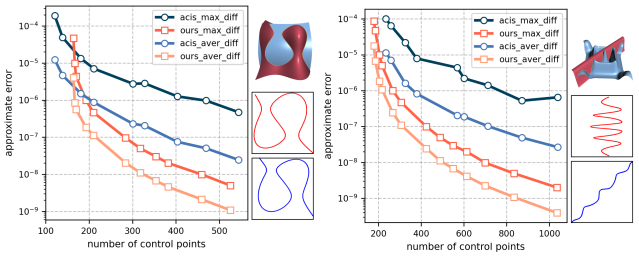
Table 3: Statistics for the parametric surface intersection examples in Fig 6. The number pairs in the first column indicate which row and column of Fig 6 the data refers to. Max_diff ($\times 10^{-6}$) indicates the maximum distance between the images of intersection curves in parametric domains estimated by sampling. Num_cp is the number of control points; Num_tp is the number of tracing points.

Examples	Ours			ACIS			OCCT		
	Max_diff	Time (s)	Num_cp	Max_diff	Time(s)	Num_cp	Max_diff	Time(s)	Num_tp
Fig 6 (1, 1)	**3.69645**	0.055	**78**	13.6933	0.011	166	3.91927	**0.002**	395
Fig 6 (1, 2)	**4.62251**	0.062	**38**	20.7854	0.005	67	6.49851	**0.002**	245
Fig 6 (1, 3)	**3.45960**	0.064	**65**	16.0494	0.021	85	4.45942	**0.003**	744
Fig 6 (2, 1)	**2.33364**	0.126	**98**	3.75187	0.014	238	7.37615	**0.005**	2200
Fig 6 (2, 2)	**35.2748**	0.325	**75**	2879.89	**0.004**	118	42.3922	96.98	79291
Fig 6 (2, 3)	**3.02551**	0.175	144	172.981	0.025	**104**	3.40048	**0.004**	1573
Fig 6 (3, 1)	**26.9667**	1.120	**185**	27.2993	0.019	485	36.5977	**0.007**	1885
Fig 6 (3, 2)	**1.04780**	0.275	**240**	1.09359	0.065	831	1.31748	**0.012**	6756
Fig 6 (3, 3)	**2.16912**	0.776	752	2.17119	0.041	**720**	3.00735	**0.010**	4561
Fig 6 (4, 1)	**1.18507**	0.288	**172**	1.30108	0.113	617	1.21887	**0.016**	5436
Fig 6 (4, 2)	**1.00013**	0.445	**223**	1.16020	0.231	638	1.50262	**0.008**	4353
Fig 6 (4, 3)	**5.44890**	0.578	**439**	7.81989	0.518	832	5.62967	**0.016**	3988
Mean	**7.51919**	0.357	**209.08**	262.333	**0.089**	408.42	9.77664	8.089	9285.58

curve. The parameter *fitol* controls the accuracy of the interpolation curves. When *fitol* is reduced to 10^{-6}, further reduction does not continue to improve accuracy. In OCCT, surface intersection is performed using the *IntPatch_Intersection* class, which outputs a sequence of tracing points, with tolerance parameters controlling the density and stability.

To ensure a fair comparison, we configured tolerance parameters to align the accuracy of ACIS and OCCT, and set the precision of our method to be no lower than theirs. Under these conditions, the three methods are compared in terms of efficiency and the number of data points required to achieve that accuracy. Table 3 displays the comparison results, demonstrating that our method achieves higher precision with a reduced number of control points. Overall, OCCT yields the largest number of data points, as it linearly approximates the intersection curves. However, our method first approximates the intersection curves using refined topological contours and then employs cubic curves for further optimization. ACIS also utilizes cubic B-spline curves to model intersection curves. The reason for the relatively higher control point count of ACIS might be that it improves geometric accuracy by recursively subdividing the interpolated curves, which anchors an excessive number of data points to the intersection geometry, thereby limiting the curve's degrees of freedom for adaptive adjustment.

Our average computation time is approximately four times that of ACIS because we perform a careful topology determination to ensure the correctness of the results. It is noted that OCCT spends significantly more time on the example Fig 6 (2,2). This could be attributed to the fact that the intersection curve coincides with the self-intersection curve of one of the surfaces. The singularity of the surface poses substantial challenges to the tracing process. In other cases, OCCT exhibits high efficiency, with an average computation time of 0.008 seconds.



(a) (b)

Figure 7: Maximum and average approximation errors under different numbers of control points for our algorithm and ACIS. The two intersected surfaces and the corresponding two p-curves for each test case are presented on the right of (a) and (b).

To further investigate the relationship between precision and the number of control points, we conducted experiments on our algorithm and ACIS with multiple tolerance values, obtaining the variation of approximate errors with the number of control points, as shown in Fig 7. Overall, as the number of control points increases, the approximation errors of both methods decrease, and our method exhibits a faster reduction. With the same number of control points, our method achieves a precision approximately two orders of magnitude higher than ACIS. This result demonstrates our method's significant advantages in control point utilization efficiency and geometric approximation accuracy.

5.3. Robustness

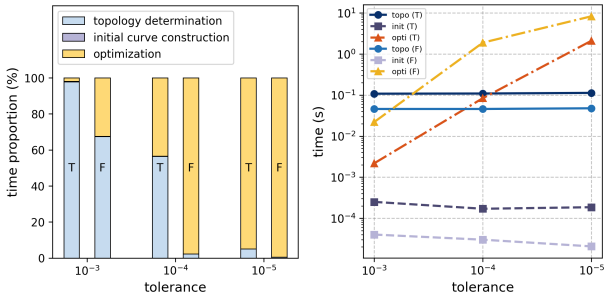
To validate the robustness of our method, we performed a series of experiments on high-degree surfaces and intersection cases with multiple loops.

5.3.1. Comparison of equation-solving methods in topology determination

In the topology determination step, we employ sampling and Newton iteration to solve a series of nonlinear equation systems and introduce an error-detection mechanism to avoid solution omission. Theoretically, algebraic methods can yield all solutions exactly. However, they suffer from the combinatorial explosion in computational complexity as the degree of surfaces increases. To substantiate this, We select surfaces with bi-degrees ranging from 2 to 7 and compared the performance of the algebraic method and our method in solving turning points (Eq.4), a single step within the topology determination process. We adopt the *solve* function of Maple [Map20] (with output precision set to 10^{-10}) as a representative algebraic method. The results are listed in Table 4. The algebraic method takes 2.55s for the bi-quadratic case and over 600s for higher degrees, making it infeasible for full topology determination process. In contrast, our method demonstrates superior efficiency. For the bi-quadratic case, the difference between our results and those of the algebraic method is less than 10^{-6}, confirming the accuracy of our approach.

5.3.2. Intersection of high-degree surfaces

To evaluate the performance of our method on high-degree surfaces, we generate 100 random cases, with each case consisting



(a) (b)

Figure 8: Performance of our method on bi-degree 5 to 7 surfaces under different tolerances, with "T" denoting the topology determination process including refinement and "F" indicating the absence of such refinement. (a) and (b) show the time proportion and values respectively.

Table 4: Comparative results of the algebraic method and our method for solving turning points of surfaces with distinct degrees.

Degree	Alge_Turn (s)	Ours_Turn/ Ours_Topo (s)	Diff
2	2.55	0.003 / 0.013	$< 10^{-6}$
3	> 600	0.002 / 0.023	\
4	> 600	0.002 / 0.031	\
5	> 600	0.003 / 0.043	\
6	> 600	0.002 / 0.048	\
7	> 600	0.003 / 0.041	\

of two surfaces of bi-degree 5 to 7. Our method achieves success in all cases. The statistical results under different tolerances and with or without topology refinement are illustrated in Fig 8. In the tolerance of 10^{-3}, topology determination accounts for the majority of the time. The initial curve constructed from the refined topological contour already nearly satisfies the tolerance. Even without refinement, only a few optimization iterations are required. As the tolerance decreases to 10^{-4}, the discrepancy between algorithms with and without refinement becomes more pronounced. When the tolerance is reduced further to 10^{-5}, optimization dominates the total computation time. The refinement process introduces $10.1\times$, $22.5\times$ and $3.9\times$ improvements in optimization time, along with $0.6\times$, $10.0\times$ and $3.8\times$ improvements in total time, under tolerances of 10^{-3}, 10^{-4} and 10^{-5}, respectively. The reason why the magnitude of the improvement for 10^{-5} is smaller than that for 10^{-4} lies in the maximum number of optimization iterations we set (10^3), which limits the potential increase in optimization time. We set the refinement threshold $\delta = 0.05$ for all test cases and the approximate error of the initial curves ranges from 10^{-3} to 10^{-4}. Future work will investigate the adjustment of this threshold based on tolerance and surface geometry to further enhance performance.

5.3.3. Intersection with multiple loops

To further demonstrate the effectiveness of our algorithm on complex cases, we select several examples featuring multiple loops. Fig 9 presents the examples with 24, 32 and 40 loops, and Fig 10

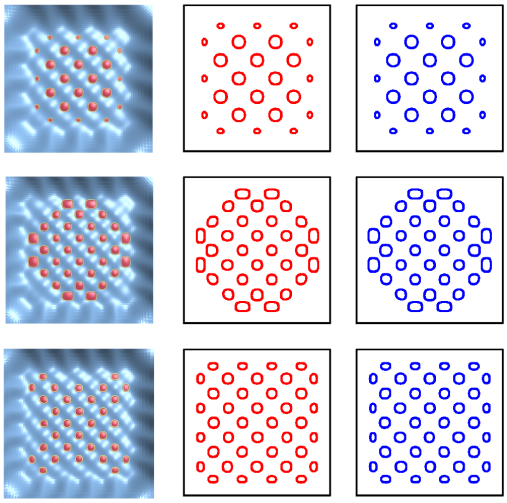


Figure 9: Parametric surface/surface intersection examples with 24/32/40 loops.

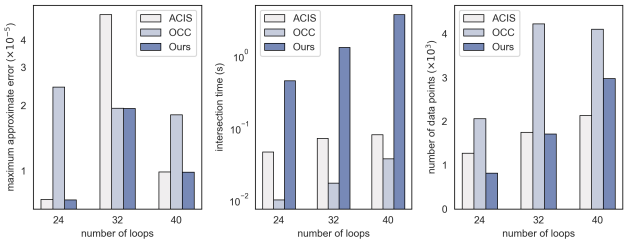


Figure 10: Statistics for parametric surface intersection with multiple loops illustrated in Fig 9.

provides comparative data against OCCT and ACIS in terms of accuracy, computational efficiency and the number of control points. For multi-branch intersection scenarios, detecting all branches is challenging, yet essential, for correct intersection calculation. Our method employs numerical approaches for topology determination, which inherently carries the risk of overlooking loops. Observations of the algorithm's execution process indicate that a sampling density 15×15 is sufficient to generate the correct topological structure for the first example (24 loops). In contrast, this density leads to the omission of turning points in the other two cases. Such omissions are identified during the subsequent connectivity computation step, triggering an automatic increase in the sampling density to 25×25 to ensure the correct results. The error-detection strategy ensures the robustness of our method, albeit with an increase in runtime.

With the approximation error around 10^{-5}, both the computation time and the number of data points required for the intersection results increase with the number of loops. Compared with OCCT and ACIS, our method can achieve the highest accuracy with relatively fewer data points. Our method consumes more computational time, yet remains applicable. The added time is partly due to the error-detection process in the topology determination step. Another reason is that we split each loop into segments and treat them sequentially. In practical applications, parallelization could be employed to improve efficiency.

6. Conclusions

We present a practical algorithm for computing parametric surface/surface intersection, focusing on the consistency of approximate intersection curves in parametric domains. The algorithm begins with topology analysis of the intersection by solving lower dimensional equations. Numerical iteration combined with error-detection is used to raise efficiency without losing stability. The topological contours are then adaptively refined to improve their proximity to the intersection. Subsequently, initial approximate curves are constructed via interpolation based on the refined contours. Finally, constrained optimization combined with knot insertion is utilized to maximize the consistency of the approximate curves, thereby minimizing the gaps between the intersecting surfaces. We have demonstrated the topological correctness of our method on various examples of parametric surfaces intersection with representative topologies. Additionally, a detailed comparison of the accuracy and output data volume of our algorithm with the intersection functions in ACIS and OCCT is presented, which indicates that our method can achieve high accuracy with relatively fewer data points and under computationally practical time. Furthermore, experiments conducted on multi-loop scenarios validate the efficacy of the error-detection mechanism and the robustness of our algorithm.

The limitation of our method is due to the drawback of the sampling approach used in topology determination. Characteristic points lying very close to each other may be overlooked under sparse sampling. Our automatic error-detection and sampling density enhancement strategy addresses this issue, whereas global sampling introduces unnecessary searches and excessive runtime. Integrating octree-based search for localizing the solution region with algebraic analysis of intersection properties could yield improvements. Since the computation of each characteristic point is conducted separately and each pair of B-spline curves is optimized independently, CPU or GPU parallelization can be applied to further enhance efficiency.

Acknowledgement

This work is partially supported by the National Key R&D Program of China (2024YFE0204400), the National Natural Science Foundation for Young Scholars (Type A) (12525114), the Key Project of the National Natural Science Foundation of China (12494550 and 12494551), and the Strategic Priority Research Program of the Chinese Academy of Sciences (XDB0640000 and XDB0640200).

References

[BFJP87] BARNHILL R., FARIN G., JORDAN M., PIPER B.: Surface/surface intersection. *Computer Aided Geometric Design 4*, 1 (1987), 3–16. 1, 2

[BHLH88] BAJAJ C., HOFFMANN C., LYNCH R., HOPCROFT J.: Tracing surface intersections. *Computer Aided Geometric Design 5*, 4 (1988), 285–307. 3

[BK90] BARNHILL R., KERSEY S.: A marching method for parametric surface/surface intersection. *Computer Aided Geometric Design 7*, 1 (1990), 257–280. 1, 3

[BLZ24] BO P., LIU Q., ZHANG C.: Topological structure extraction for computing surface–surface intersection curves. *The Visual Computer 41*, 5 (2024), 3503–3518. 2, 3, 7

[Boo01] BOOR C. D.: *A Practical Guide to Splines*, 1 ed. Applied Mathematical Sciences. Springer New York, NY, 2001. 6

[CZXL23] CHENG J.-S., ZHANG B., XIAO Y., LI M.: Topology driven approximation to rational surface-surface intersection via interval algebraic topology analysis. *ACM Transactions on Graphics 42*, 4 (2023), 1–16. 2, 3

[Dok01] DOKKEN T.: *Approximate implicitization*, 1 ed. Vanderbilt University, USA, 2001, p. 81–102. 2

[DSY89] DOKKEN T., SKYTT V., YTREHUS A.-M.: Recursive subdivision and iteration in intersections and related problems. In *Mathematical Methods in Computer Aided Geometric Design*, LYCHE T., SCHUMAKER L. L., (Eds.). Academic Press, 1989, pp. 207–214. 3

[GK97] GRANDINE T. A., KLEIN F. W.: A new approach to the surface intersection problem. *Computer Aided Geometric Design 14*, 2 (1997), 111–134. 2, 3, 4

[HEFS85] HOUGHTON E. G., EMNETT R. F., FACTOR J. D., SABHARWAL C. L.: Implementation of a divide-and-conquer method for intersection of parametric surfaces. *Computer Aided Geometric Design 2*, 1 (1985), 173–183. 2

[HFH*07] HASS J., FAROUKI R. T., HAN C. Y., SONG X., SEDERBERG T. W.: Guaranteed consistency of surface intersections and trimmed surfaces using a coupled topology resolution and domain decomposition scheme. *Advances in Computational Mathematics 27* (2007), 1–26. 2

[HjOwK09a] HUR S., JAE OH M., WAN KIM T.: Approximation of surface-to-surface intersection curves within a prescribed error bound satisfying G2 continuity. *Computer-Aided Design 41*, 1 (2009), 37–46. 2, 3

[HjOwK09b] HUR S., JAE OH M., WAN KIM T.: Classification and resolution of critical cases in grandine and klein's topology determination using a perturbation method. *Computer Aided Geometric Design 26*, 2 (2009), 243–258. 2, 3

[JCY22] JIA X., CHEN F., YAO S.: Singularity computation for rational parametric surfaces using moving planes. *ACM Transactions on Graphics 42*, 1 (2022), 1–14. 4

[JLC22] JIA X., LI K., CHENG J.: Computing the intersection of two rational surfaces using matrix representations. *Computer-Aided Design 150* (2022), 103303. 1, 2

[KA10] KO K. H., AHN H. S.: Approximation of 3d surface-to-surface intersection curves. *Engineering with Computers 26* (2010), 49–60. 2, 3

[KM97] KRISHNAN S., MANOCHA D.: An efficient surface intersection algorithm based on lower-dimensional formulation. *ACM Transactions on Graphics 16* (1997), 74–106. 2, 3

[KPW92] KRIEZIS G., PATRIKALAKIS N., WOLTER F.-E.: Topological and differential-equation methods for surface intersections. *Computer-Aided Design 24*, 1 (1992), 41–55. 3

[KS88] KATZ S., SEDERBERG T. W.: Genus of the intersection curve of two rational surface patches. *Computer Aided Geometric Design 5*, 3 (1988), 253–258. 1

[KS12] KERBER M., SAGRALOFF M.: A worst-case bound for topology computation of algebraic curves. *Journal of Symbolic Computation 47*, 3 (2012), 239–258. 3, 5

[LQLX14] LIN H., QIN Y., LIAO H., XIONG Y.: Affine arithmetic-based b-spline surface intersection with gpu acceleration. *IEEE Trans. Vis. Comput. Graphics 20*, 2 (2014), 172–181. 1, 2

[Map20] MAPLESOFT, A DIVISION OF WATERLOO MAPLE INC.: Maple. https://www.maplesoft.com, 2020. Accessed: 2025-06-04. 10

[ML95] MA Y., LUO R. C.: Topological method for loop detection of surface intersection problems. *Computer-Aided Design 27*, 11 (1995), 811–820. 3

[ML98] MA Y., LEE Y.-S.: Detection of loops and singularities of surface intersections. *Computer-Aided Design 30*, 14 (1998), 1059–1067. 3

[PSKE20] PARK Y., SON S.-H., KIM M.-S., ELBER G.: Surface–surface-intersection computation using a bounding volume hierarchy with osculating troidal patches in the leaf nodes. *Computer-Aided Design 127* (2020), 102866. 2

[PT96] PIEGL L., TILLER W.: *The NURBS Book*, 2 ed. Monographs in Visual Communication. Springer Berlin, Heidelberg, 1996. 6

[Sar83] SARRAGA R. F.: Algebraic methods for intersections of quadric surfaces in gmsolid. *Computer Vision, Graphics, and Image Processing 22*, 2 (1983), 222–238. 2

[SAS25] SAS O. C.: Open CASCADE technology. https://dev.opencascade.org/, 2025. Accessed: 2025-06-04. 7

[SC95] SEDERBERG T. W., CHEN F.: Implicitization using moving curves and surfaces. In *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques* (New York, NY, USA, 1995), SIGGRAPH '95, Association for Computing Machinery, pp. 301–308. 2

[SFL*08] SEDERBERG T. W., FINNIGAN G. T., LI X., LIN H., IPSON H.: Watertight trimmed NURBS. *ACM Transactions on Graphics 27*, 3 (2008), 1–8. 3

[Sky05] SKYTT V.: Challenges in surface-surface intersections. In *Computational Methods for Algebraic Spline Surfaces* (Berlin, Heidelberg, 2005), Springer Berlin Heidelberg, pp. 11–26. 2

[SM88] SEDERBERG T. W., MEYERS R. J.: Loop detection in surface patch intersections. *Computer Aided Geometric Design 5*, 2 (1988), 161–171. 3

[SN91] SEDERBERG T. W., NISHITA T.: Geometric hermite approximation of surface patch intersection curves. *Computer Aided Geometric Design 8*, 2 (1991), 97–114. 2, 3

[SSZ*04] SONG X., SEDERBERG T. W., ZHENG J., FAROUKI R. T., HASS J.: Linear perturbation methods for topologically consistent representations of free-form surface intersections. *Computer Aided Geometric Design 21*, 3 (2004), 303–319. 3

[Tea25] TEAM S.: 3D ACIS Modeler. https://www.spatial.com/products/3d-acismodeling, 2025. Accessed: 2025-06-04. 7

[The25] THE CGAL PROJECT: CGAL, Computational Geometry Algorithms Library. https://www.cgal.org, 2025. Accessed: 2025-06-04. 7

[UMC*19] URICK B., MARUSSIG B., COHEN E., CRAWFORD R. H., HUGHES T. J., RIESENFELD R. F.: Watertight boolean operations: A framework for creating cad-compatible gap-free editable solid models. *Computer-Aided Design 115* (2019), 147–160. 3

[YJY23] YANG J., JIA X., YAN D.-M.: Topology guaranteed b-spline surface/surface intersection. *ACM Transactions on Graphics 42*, 6 (2023), 1–16. 1, 2, 3

[YKK01] YAMAGUCHI Y., KAMIYAMA R., KIMURA F.: *Surface-Surface Intersection with Critical Point Detection Based on Bézier Normal Vector Surfaces*. Springer US, Boston, MA, 2001, pp. 287–308. 3